CT-159 · DATA STRUCTURES

Sorting Algorithms

Lecture 6 — Selection, Bubble & Insertion Sort

Presenter: Dr. Muhammad Kamran

SECTION 01

Sorting Basics

Arranging data in order — and the vocabulary to describe how.

SECTION 01 · BASICS

01

What is Sorting?

Sorting arranges data in a particular order — usually ascending or descending.

- Searching becomes far faster on sorted data (e.g. binary search needs it).
- Sorting also makes data easier to read, compare, and present.
- Many algorithms use sorting as a first step.

SECTION 01 · BASICS

02

In-place vs Not-in-place

In-place	Sorts within the array itself, using no (or O(1)) extra space. e.g. Bubble, Selection, Insertion.
Not-in-place	Needs extra space ≥ the number of elements being sorted. e.g. standard Merge Sort.

SECTION 01 · BASICS

03

Adaptive vs Non-adaptive

Adaptive	Takes advantage of elements already in order — faster on nearly sorted data. e.g., insertion sort.
Non-adaptive	Ignores existing order; does the same work regardless. e.g. Selection Sort.

SECTION 02

Selection Sort

Repeatedly pick the smallest and place it at the front.

SECTION 02 · SELECTION

04

Selection Sort

Repeatedly find the minimum of the unsorted part and move it to the front.

- An in-place, comparison-based sort.
- The list is split into a sorted part (left) and an unsorted part (right).
- Each pass grows the sorted part by one element.



SECTION 02 · SELECTION

05

Selection Sort — Algorithm

Steps

1. Set MIN to the first unsorted index
2. Scan the rest to find the true minimum
3. Swap that minimum into MIN's place
4. Move the sorted boundary right by one
5. Repeat until the array is sorted

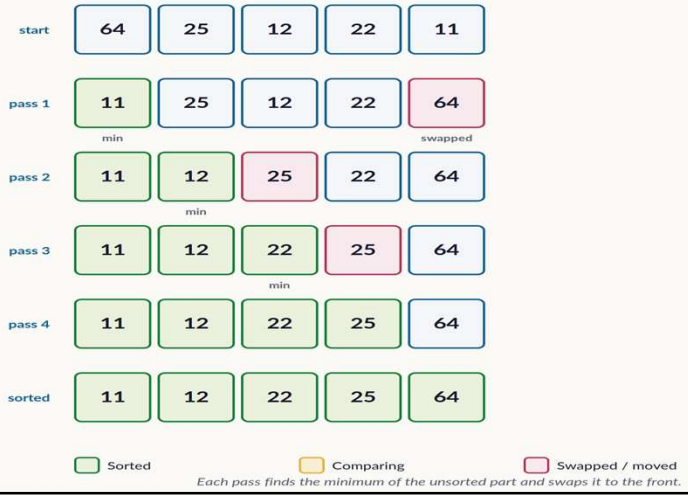
```
// pseudocode
for i = 0 to n-2:
    min = i
    for j = i+1 to n-1:
        if arr[j] < arr[min]:
            min = j
    swap(arr[i], arr[min])
```

SECTION 02 · SELECTION

06

Selection Sort — Worked Example

Sorting [64, 25, 12, 22, 11] — each pass selects the minimum and swaps it forward.



SECTION 02 · SELECTION

07

Selection Sort — Complexity

Best	Average	Worst	Space
$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

Always scans the whole unsorted part, so it's $O(n^2)$ even on sorted input (non-adaptive). It does the fewest swaps of the three, though — at most $n-1$.

SECTION 03

Bubble Sort

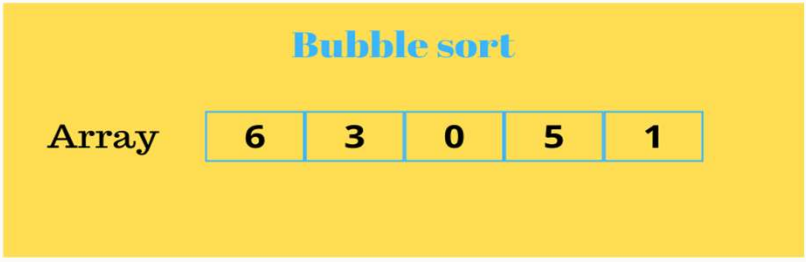
Swap adjacent out-of-order pairs until nothing moves.

SECTION 03 · BUBBLE

Bubble Sort

Compare each adjacent pair and swap them if they're out of order.

- A comparison-based, in-place sort.
- Each pass 'bubbles' the largest remaining element to the end.
- Simple, but $O(n^2)$ — not suited to large data sets.



SECTION 03 · BUBBLE

09

Bubble Sort — Algorithm

Steps

1. Loop over the list
2. Compare each adjacent pair
3. If they're out of order, swap them
4. After each pass, the largest is at the end
5. Repeat for the remaining elements

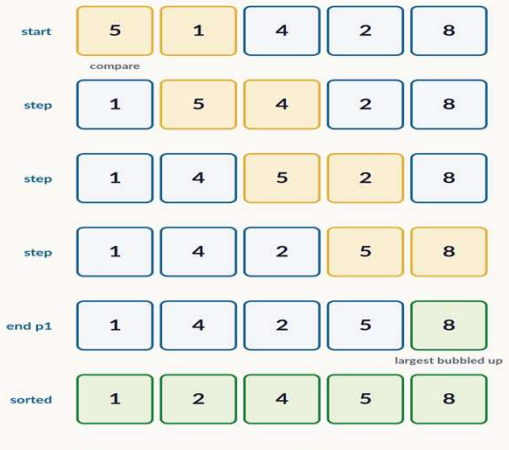
```
// pseudocode
begin BubbleSort(list)
  for i = 0 to n-1:
    for j = 0 to n-2-i:
      if list[j] > list[j+1]:
        swap(list[j], list[j+1])
  return list
end BubbleSort
```

SECTION 03 · BUBBLE

10

Bubble Sort — Worked Example

First pass on [5, 1, 4, 2, 8] — the largest element bubbles to the end.



Adjacent pairs are compared and swapped; the largest element 'bubbles' to the end each pass.

SECTION 03 · BUBBLE

Bubble Sort — Complexity

Best	Average	Worst	Space
O(n)	O(n²)	O(n²)	O(1)

Best case O(n) only with the optimized version on already-sorted data (one clean pass). Average and worst remain O(n²).

SECTION 04

Insertion Sort

Build the sorted list one element at a time.

SECTION 04 · INSERTION

13

Insertion Sort

Take each element and insert it into its correct place among the already-sorted ones.

- An in-place, comparison-based, adaptive sort.
- Works the way people sort playing cards in their hand.
- Efficient for small or nearly-sorted data sets.

SECTION 04 · INSERTION

14

Insertion Sort — Algorithm

Steps

1. The first element is already sorted
2. Pick the next element (the key)
3. Compare it leftwards through the sorted part
4. Shift bigger elements one place right
5. Drop the key into the gap; repeat

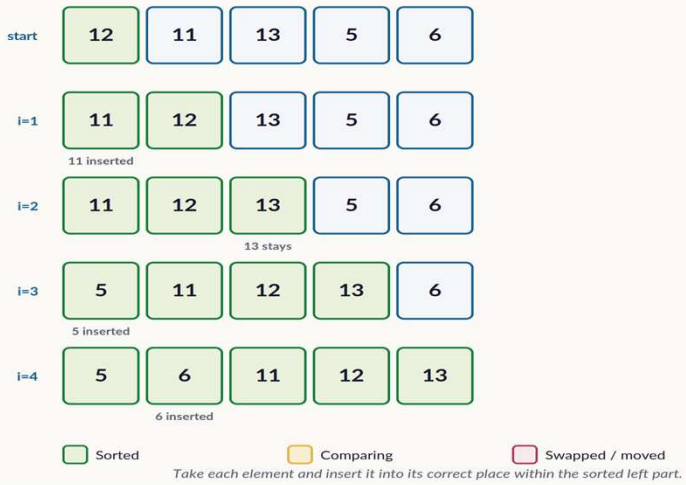
```
// pseudocode
for i = 1 to n-1:
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j = j - 1
    arr[j+1] = key
```

SECTION 04 · INSERTION

15

Insertion Sort — Worked Example

Sorting [12, 11, 13, 5, 6] — each element is inserted into the sorted left part.



Take each element and insert it into its correct place within the sorted left part.

SECTION 04 · INSERTION

16

Insertion Sort — Complexity

Best	Average	Worst	Space
O(n)	O(n²)	O(n²)	O(1)

Best case O(n) on already-sorted data (adaptive — the inner loop never shifts). Average and worst are O(n²).

SECTION 04 · INSERTION

The Three Compared

Selection	$O(n^2)$ always · fewest swaps · not adaptive
Bubble	$O(n^2)$, best $O(n)$ if optimized · adaptive · many swaps
Insertion	$O(n^2)$, best $O(n)$ · adaptive · great on nearly-sorted data

SECTION 05

Practice It Yourself

Trace each pass by hand before you code — that's how sorting clicks.

PRACTICE · SORTING

P1

Problems to Try

Write out the array after each pass on paper before touching the keyboard.

LeetCode 75	Sort Colors	Medium	Sort 0s,1s,2s in one pass (Dutch National Flag).
LeetCode 912	Sort an Array	Medium	Implement a sort from scratch (no built-ins).
LeetCode 88	Merge Sorted Array	Easy	Merge two sorted arrays in place.
LeetCode 283	Move Zeroes	Easy	Partition-style in-place rearrangement.
Codeforces 339A	Helpful Maths	Easy	A gentle sort-the-tokens warm-up.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 75 — The Approach

Problem (in short): sort an array of only 0s, 1s and 2s in a single pass, in place.

- Keep three pointers: low (next spot for 0), mid (current), high (next spot for 2).
- If arr[mid] is 0 → swap with low, advance both low and mid.
- If arr[mid] is 1 → it's already in the middle → just advance mid.
- If arr[mid] is 2 → swap with high, move high left (don't advance mid yet).

YOUR TURN

On paper, run [2, 0, 2, 1, 1, 0] through the three pointers. Where do low, mid, high end up?

PRACTICE · WORKED EXAMPLE

P3

LeetCode 75 — The Code

```
void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;
    while (mid <= high) {
        if (nums[mid] == 0)
            swap(nums[low++], nums[mid++]);
        else if (nums[mid] == 1)
            mid++;
        else                          // nums[mid] == 2
            swap(nums[mid], nums[high--]);
    }
}
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

Three regions	low, mid, high keep the array split into 0s | 1s | unknown | 2s.
Why not mid++ on 2	The value swapped from high is unseen, so it must be re-examined.
Time	O(n) — a single pass, each element handled once.
Space	O(1) — sorted fully in place, no extra array.

PRACTICE · RESOURCES

P5

Where to Get Help

Watch the idea only after you've tried it — then close the video and code it yourself.

NeetCode	Sort Colors, clearly visualised. Search: "NeetCode Sort Colors" · neetcode.io/solutions/sort-colors
Abdul Bari	Sorting algorithms with dry-runs. Search: "Abdul Bari Sorting Algorithms"
William Fiset	Sorting playlist for CS students. Search: "William Fiset Sorting"

YOUR TURN

Course rule: trace each problem on paper for 20 minutes before watching anything — and never paste it into an AI.

LECTURE 6 — RECAP

What we covered

- Sorting: in-place vs not, adaptive vs not
- Selection: pick the min each pass — $O(n^2)$
- Bubble: swap adjacent pairs — $O(n^2)$, $O(n)$ optimized
- Insertion: insert into the sorted part — adaptive
- All three are $O(n^2)$ average, $O(1)$ space
- Next: faster divide-and-conquer sorts

Next: Merge Sort & Quick Sort